Security (System) Engineering with Generative AI

Homework Assignment no. 6

Student name: Brehon Parker

FAU Email: bparker2012@fau.edu

Algorithm:

I. Edit above your student name and homework number.
II. Edit below the missing information between < and >.
III. During development, for every significant task given to the agent you must:
   A. Take a screenshot of your VSCode window clearly showing the prompt you gave to the agent and the response it gave you back. Your FAU ID is the email address part excluding @fau.edu and it must be visible on the screenshot.
   B. Take a screenshot of your VSCode window clearly showing at least the top few lines of code that were generated or modified by the agent. Your FAU ID must be visible on the screenshot.
   C. Insert the screenshots into the homework Word document, along with the task prompt given to the agent pasted as regular text.
IV. When finished, convert the Word document to PDF and upload on the homework Canvas page.

Screencast Recording URL:

https://www.loom.com/share/c0d303d6cde44483b8674fa29f8042da

Github Reposity URL:

https://github.com/bparker2012-cyber/gensec-code/tree/codex/hw6-secure-notes/hw6

Application name: *Secure Notes*

Application Description: *Secure Notes is a local Flask application generated with OpenAI Codex. Separate user accounts can create, read, edit, and delete persistent private notes stored in SQLite.*

Application generation prompts

Coding agent: OpenAI Codex, Codex desktop app. The saved prompts below describe the application and security checks used for the recorded generation.

Prompt 1 Build the application

Build a small Python 3.13 Flask application named Secure Notes inside hw6. Use app.py with a create_app factory, Jinja templates, local CSS, and SQLite. Implement registration, login, logout, and create/read/edit/delete private notes. Use a clean, readable interface with a sidebar, note list, editor, clear errors, and a footer showing Brehon Parker, bparker2012@fau.edu, and Z23222679. Keep the app usable without external APIs, JavaScript, or internet services. Treat every form field, URL, and cookie as adversarial. Use Werkzeug scrypt password hashing, parameterized SQL, owner-scoped queries on every note route, Jinja autoescaping, per-session CSRF tokens on every POST including auth and logout, session clearing at auth transitions, bounded input sizes, a persistent random signing key outside Git, HttpOnly/SameSite cookies, no-cache responses, security headers, trusted loopback hosts, and debug mode off. Add a bounded SQLite-backed login/registration rate limit. Store only an opaque session token in the signed cookie and its hash server-side so logout revokes captured cookies. Run on 127.0.0.1:5006 by default, with optional Secure cookies for HTTPS. Document that note content is plaintext at rest, local HTTP is a classroom mode, and this is not a production deployment. Pin dependencies with pyproject.toml and uv.lock, and ignore databases, signing keys, virtual environments, and caches. Keep the scope small enough to explain in a five-minute screencast.

Prompt 2 Verify security and prepare the demonstration

Exercise the real application and add meaningful automated tests for private note isolation, anonymous access, CSRF rejection, stored XSS escaping, SQL injection attempts, password hashing, revoked sessions, malformed and oversized input, security headers, and rate limiting. Fix any failures without weakening the controls. Add a reproducible --demo security-check command using temporary data and Flask's test client, clearly labeled as in-process checks. Do not add an HTTP test endpoint. Supply setup/run/test instructions, a security overview with honest limitations, and a short recording script that follows the required order: git clone, prompt walkthrough, live regeneration in a fresh directory, security explanation, functionality and limitations. Preserve real evidence; never fabricate a model response, screenshot, recording URL, or submission.

Build prompt

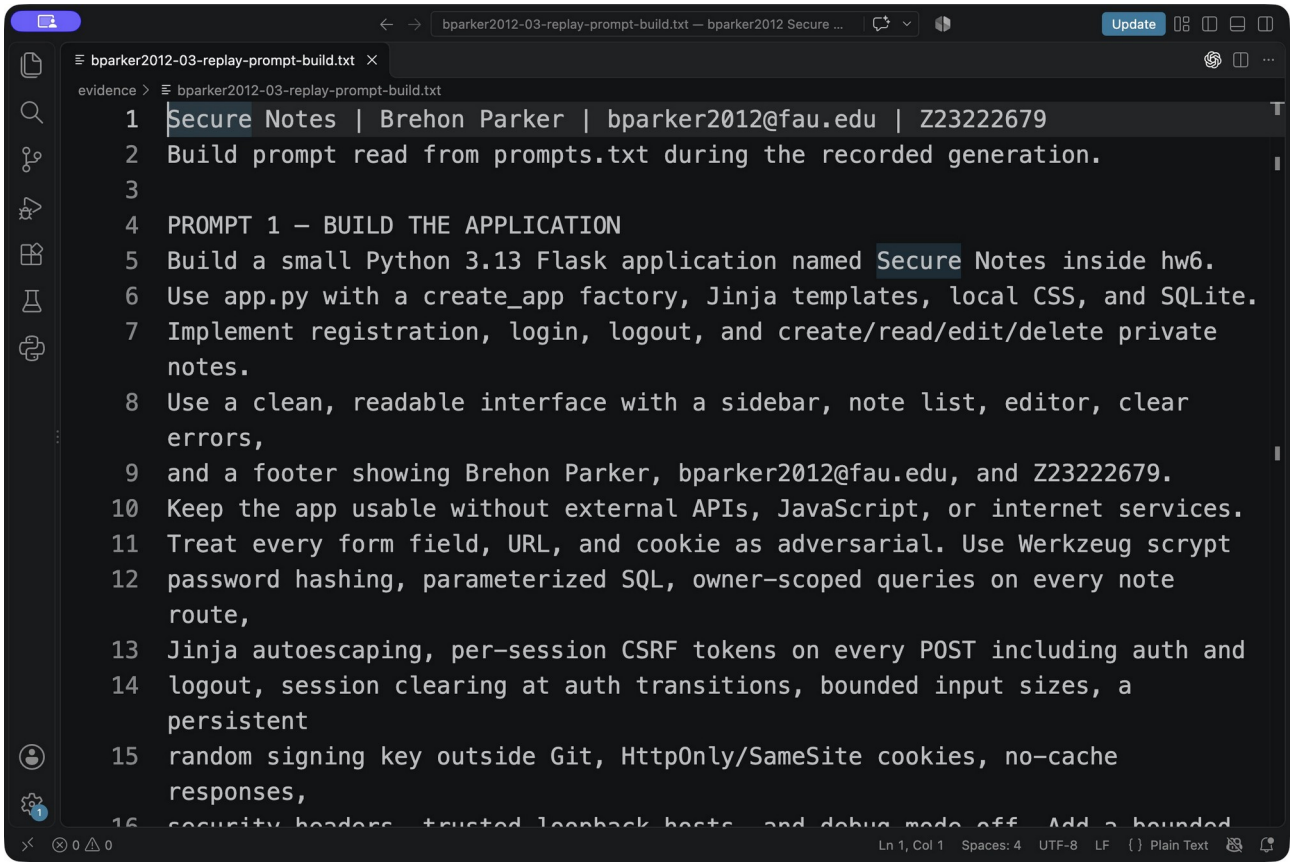


Figure 1. Application build prompt from prompts.txt, displayed in VS Code.

Build prompt continued

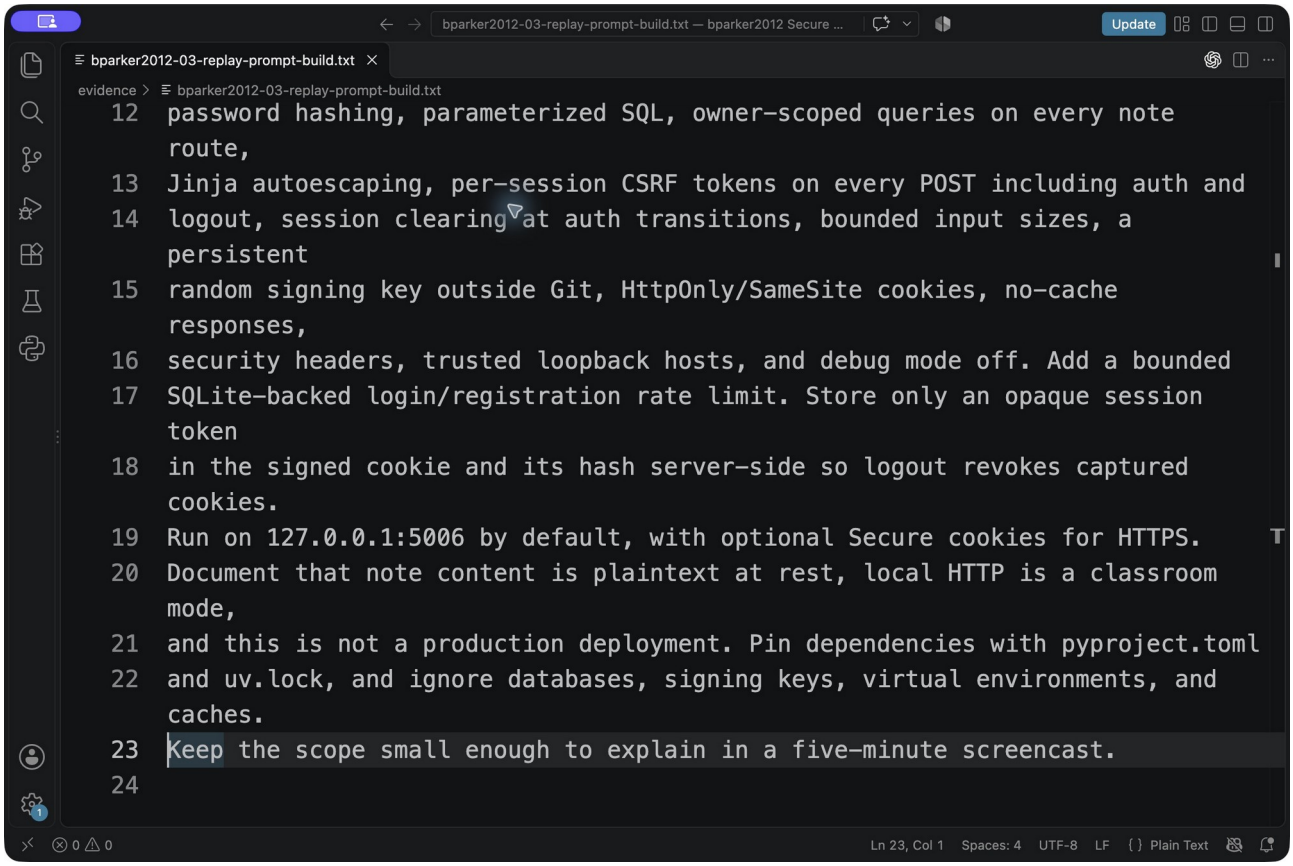


Figure 2. Remaining lines of the application build prompt.

Verification prompt

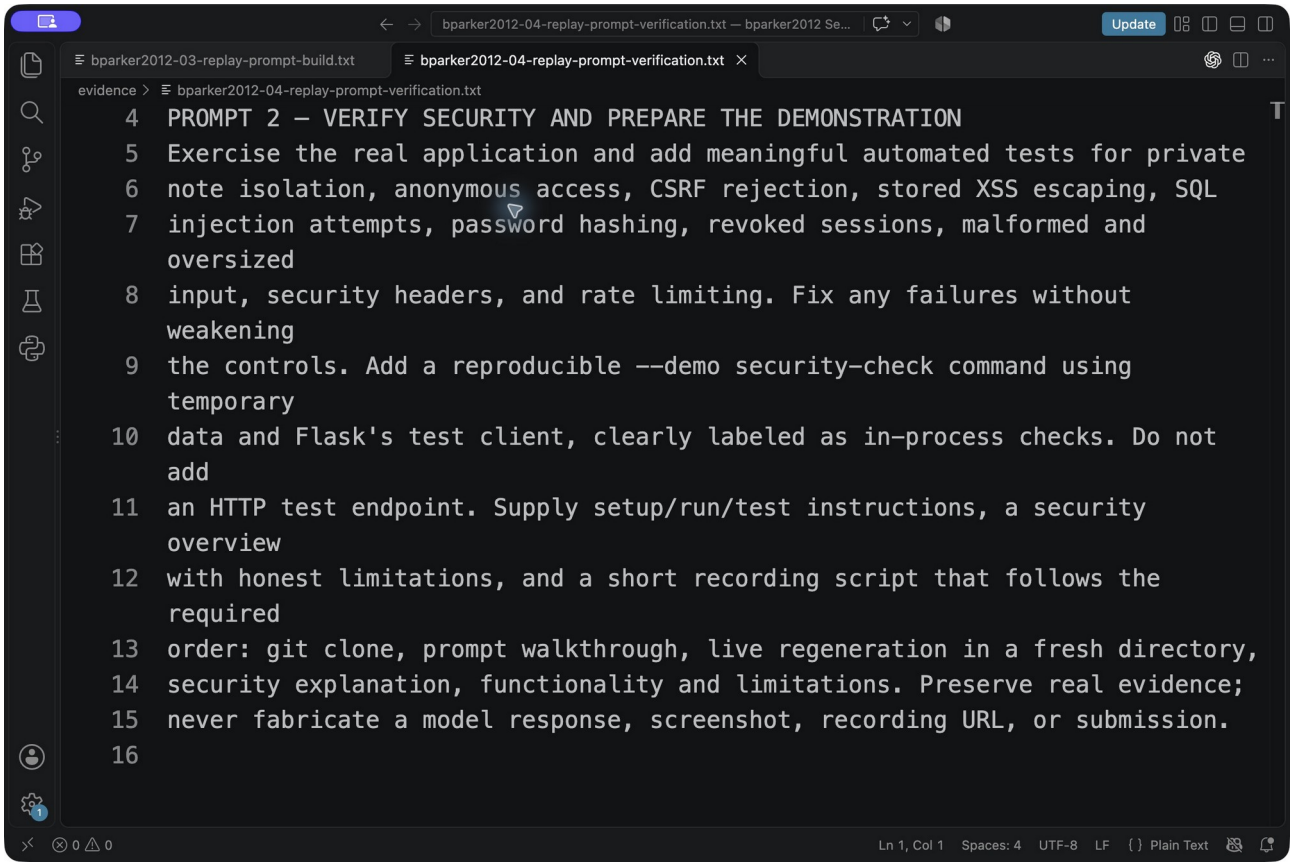


Figure 3. Security verification prompt from prompts.txt, displayed in VS Code.

Additional generation request excerpt

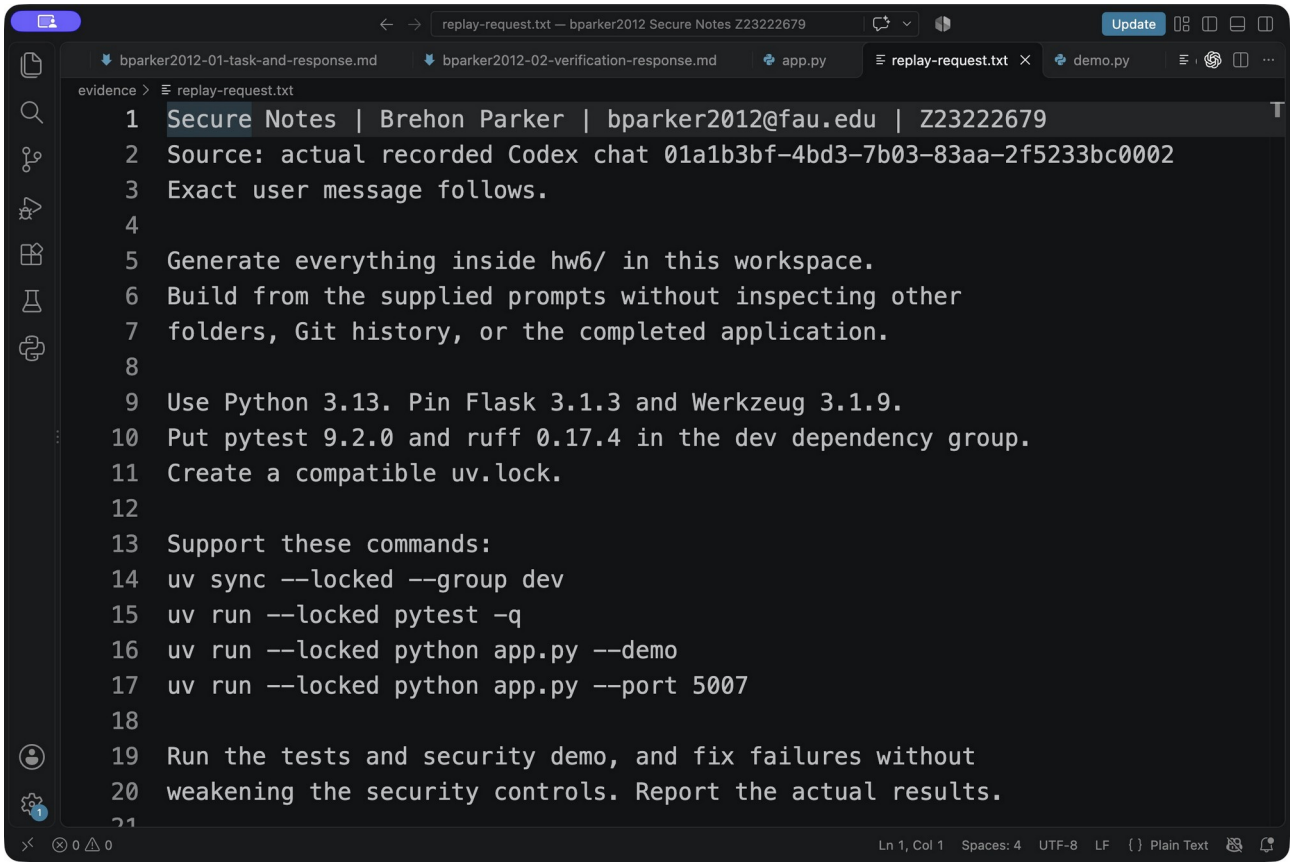


Figure 4. Technical portion of the additional version and verification request.

Generate everything inside hw6/ in this workspace.
Build from the supplied prompts without inspecting other
folders, Git history, or the completed application.

Use Python 3.13. Pin Flask 3.1.3 and Werkzeug 3.1.9.
Put pytest 9.2.0 and ruff 0.17.4 in the dev dependency group.
Create a compatible uv.lock.

Support these commands:
uv sync --locked --group dev
uv run --locked pytest -q
uv run --locked python app.py --demo
uv run --locked python app.py --port 5007

Run the tests and security demo, and fix failures without
weakening the security controls. Report the actual results.

Generated application code

Codex generated app.py, Jinja templates, local CSS, locked dependencies, and security tests. The application supports registration, login, private notes, editing, deletion, and sign-out.

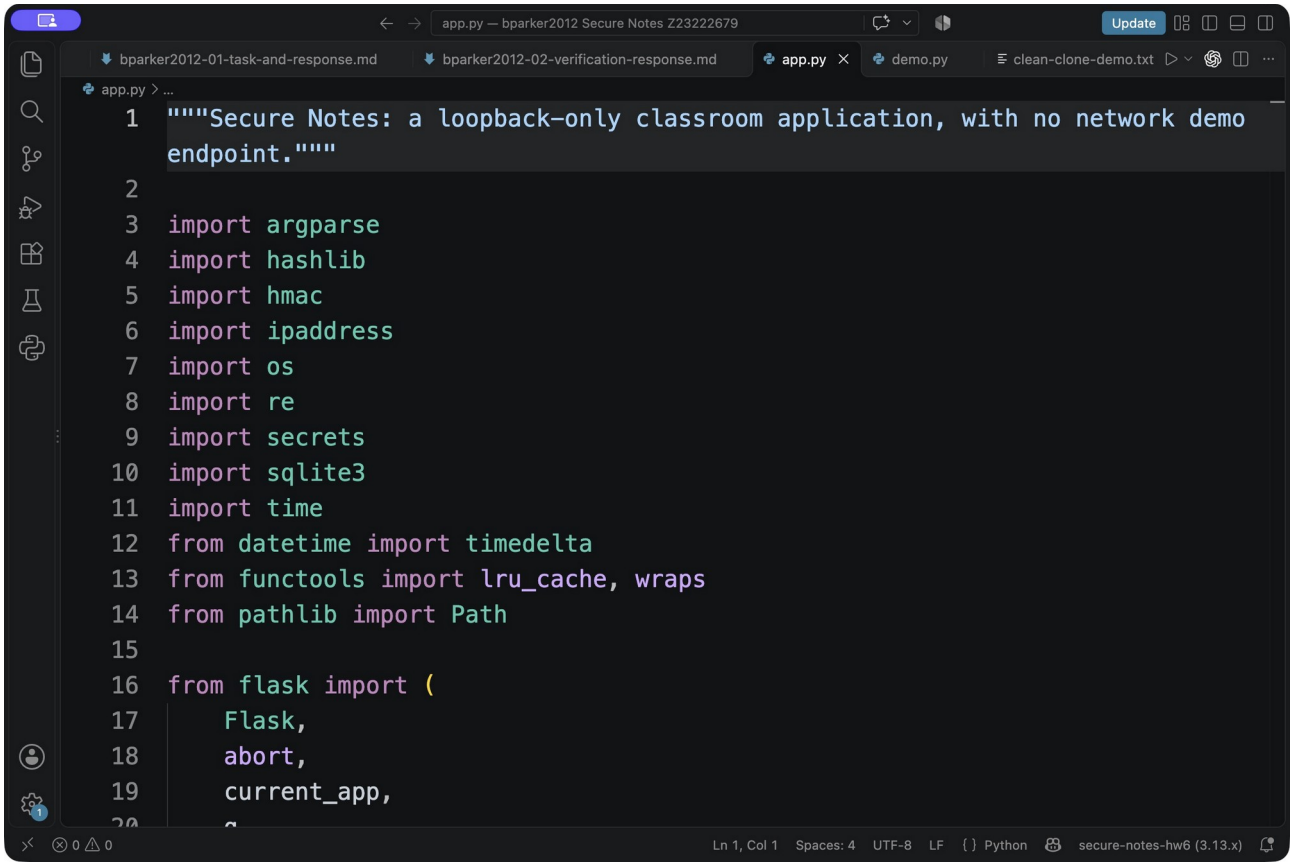


Figure 5. Top lines of app.py in VS Code, with the FAU ID visible in the window title.

Security verification and agent replies

Codex reported 120 passing tests and 13 passing security checks. Locked installation, lock validation, lint, and formatting also passed.

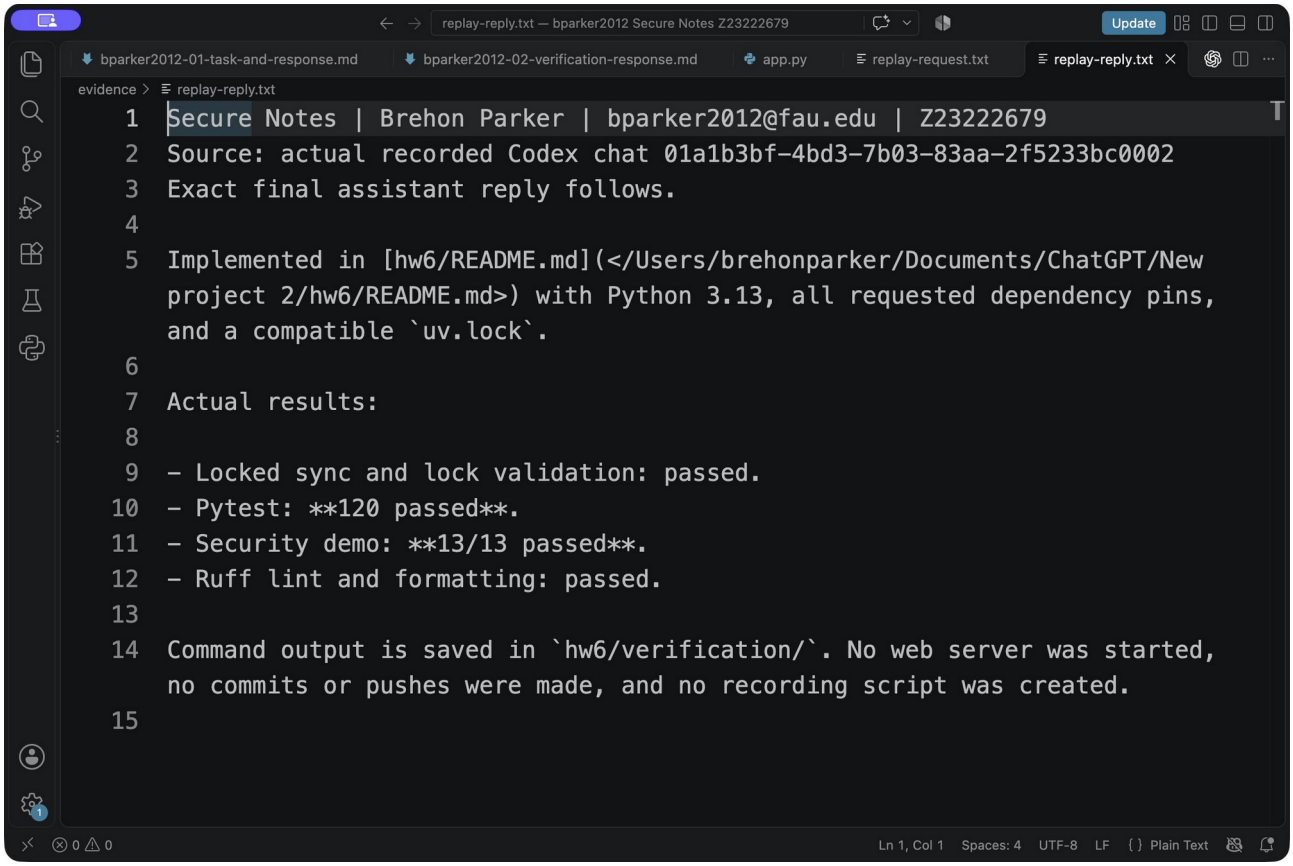


Figure 6. Final Codex reply from the recorded generation, showing the verification results.

The suite covers unauthorized note access, missing or malformed CSRF tokens, stored XSS, SQL injection, password hashes, expired or revoked sessions, oversized inputs, host rejection, persistent signing keys, bounded state, and concurrent authentication attempts. The tests exercise real routes and SQLite.

Generated verification code

The --demo command checks Flask routes and SQLite through the test client. It uses temporary test data and does not start a web server.

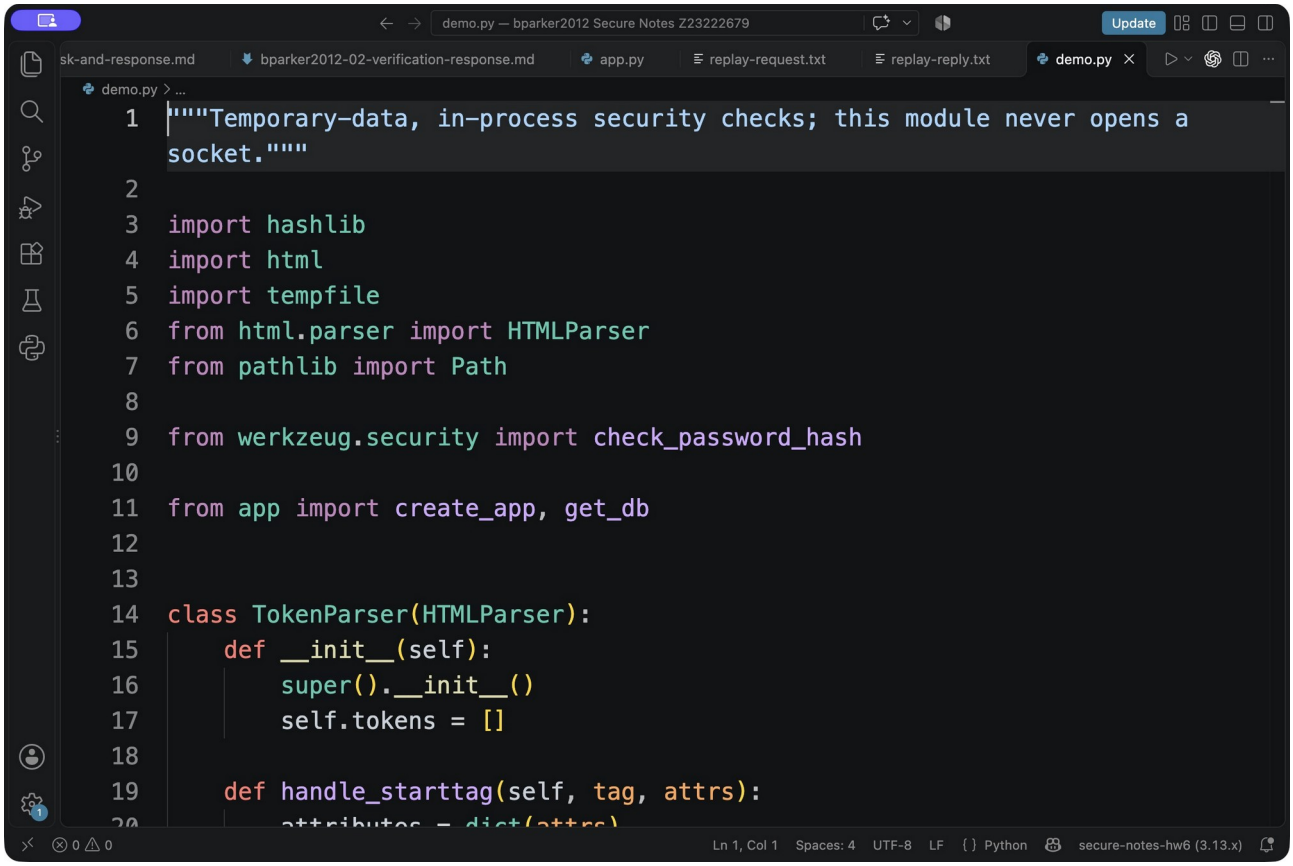


Figure 7. Top lines of demo.py in VS Code, with the FAU ID visible.

Private note ownership

Every note lookup includes both the note ID and the authenticated user ID. Update and delete statements repeat the owner condition. A second account cannot list, open, edit, or delete the first account's note; these requests are covered by the passing test suite.

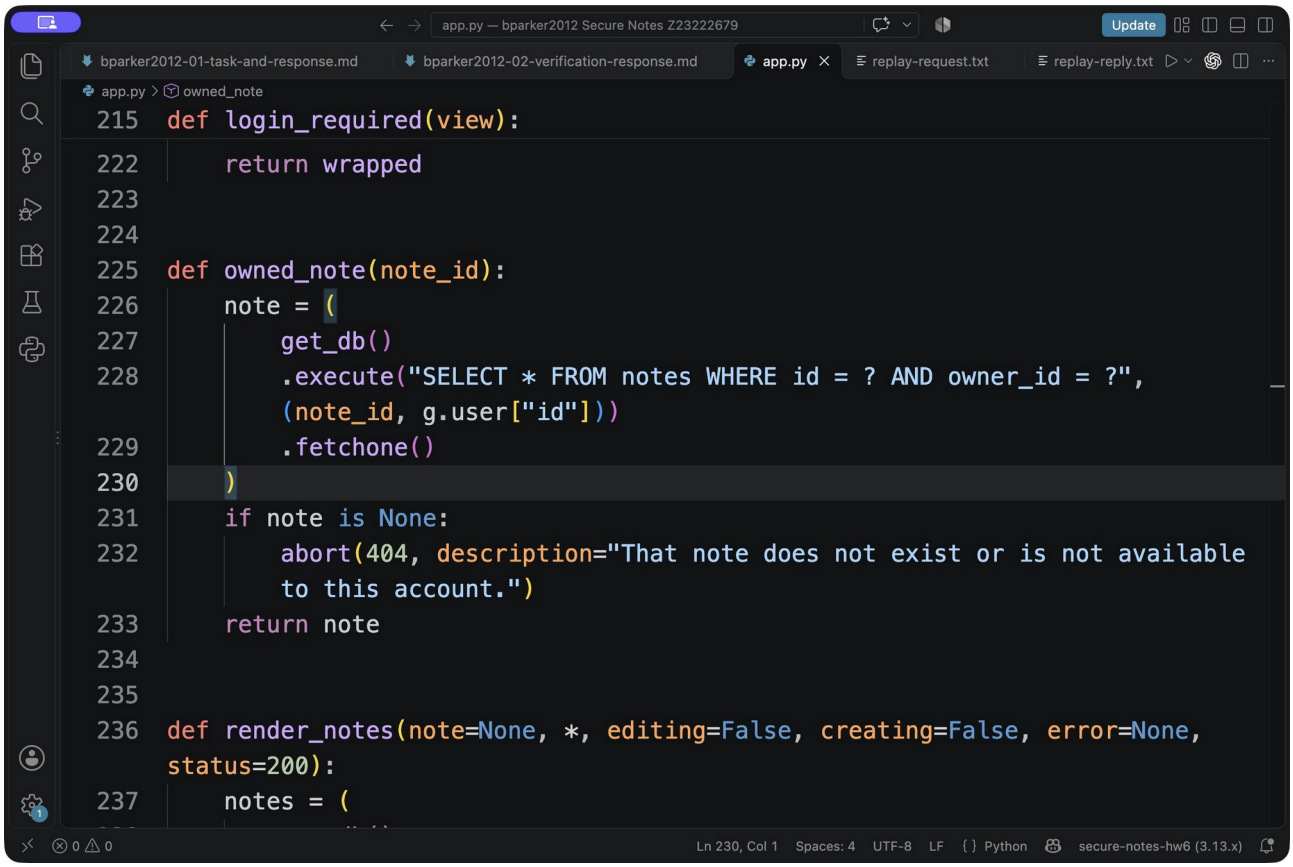


Figure 8. The note query includes the note ID and authenticated owner ID.

CSRF and authentication sessions

Every POST, including login, registration, and logout, verifies the session CSRF token. Registration, login, and logout replace the current browser's session and CSRF token. The previous session is revoked. The signed cookie contains only an opaque identifier; SQLite stores its hash and the CSRF token. Sessions expire after one hour. Cookies are HttpOnly and SameSite Strict; Secure cookies can be enabled when actual HTTPS is provided.

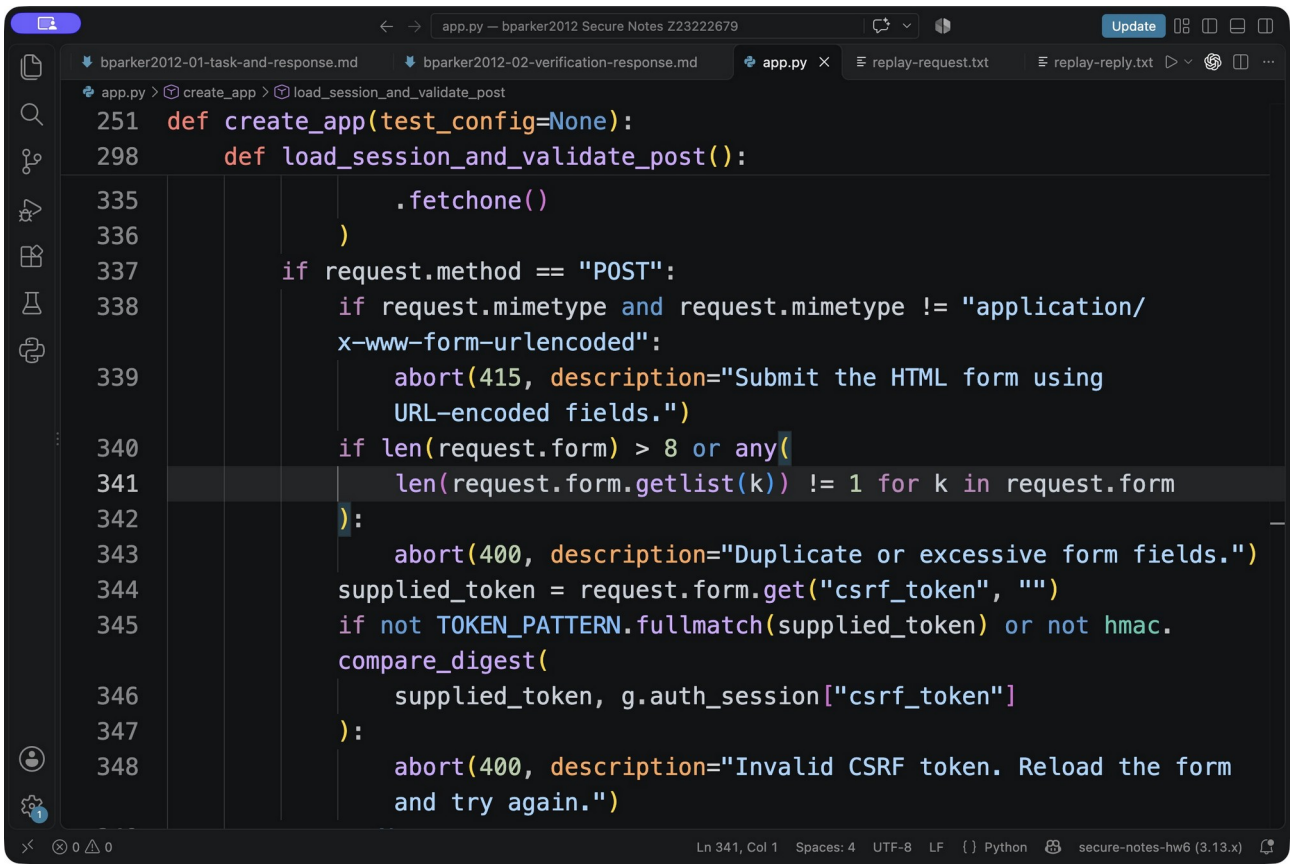


Figure 9. The request guard rejects duplicate fields and checks POST tokens.

Verification from a fresh GitHub clone

A fresh GitHub clone installed the locked dependencies and passed all 120 tests, all 13 demonstration checks, Ruff lint, and formatting. The output below comes from that check; the full command logs are saved under verification/.

```
.....................................................................  [ 60%]
.................................................  [100%]
120 passed in 8.31s
```

Actual security demonstration output

```
Secure Notes — IN-PROCESS security checks (Flask test client)
Temporary SQLite data only. No server or network listener is started.
PASS 01: registration and authentication rotate session and CSRF tokens
PASS 02: anonymous users cannot read or create notes
PASS 03: owner checks isolate note lists, reads, edits, and deletes
PASS 04: missing and cross-session CSRF tokens cannot perform actions
PASS 05: stored XSS stays escaped in the reader and textarea
PASS 06: SQL injection strings remain data and cannot bypass authentication
PASS 07: passwords use salted Werkzeug scrypt hashes
PASS 08: signed cookies contain only an opaque token; SQLite stores its hash
PASS 09: malformed fields, oversized requests, and enormous note IDs fail safely
PASS 10: security headers, no-cache responses, and trusted hosts are enforced
PASS 11: logout revokes captured cookies, including mutation attempts
PASS 12: SQLite authentication rate limits persist across app restarts
PASS 13: the owner can log back in, read, edit, and delete private notes
RESULT: 13/13 in-process security checks passed; temporary data removed.
```

Browser demonstration

Registration and note creation were checked in a browser using a temporary test account. The note reader displays the script-like input as text. The automated tests also cover reading, editing, deletion, and sign-out.

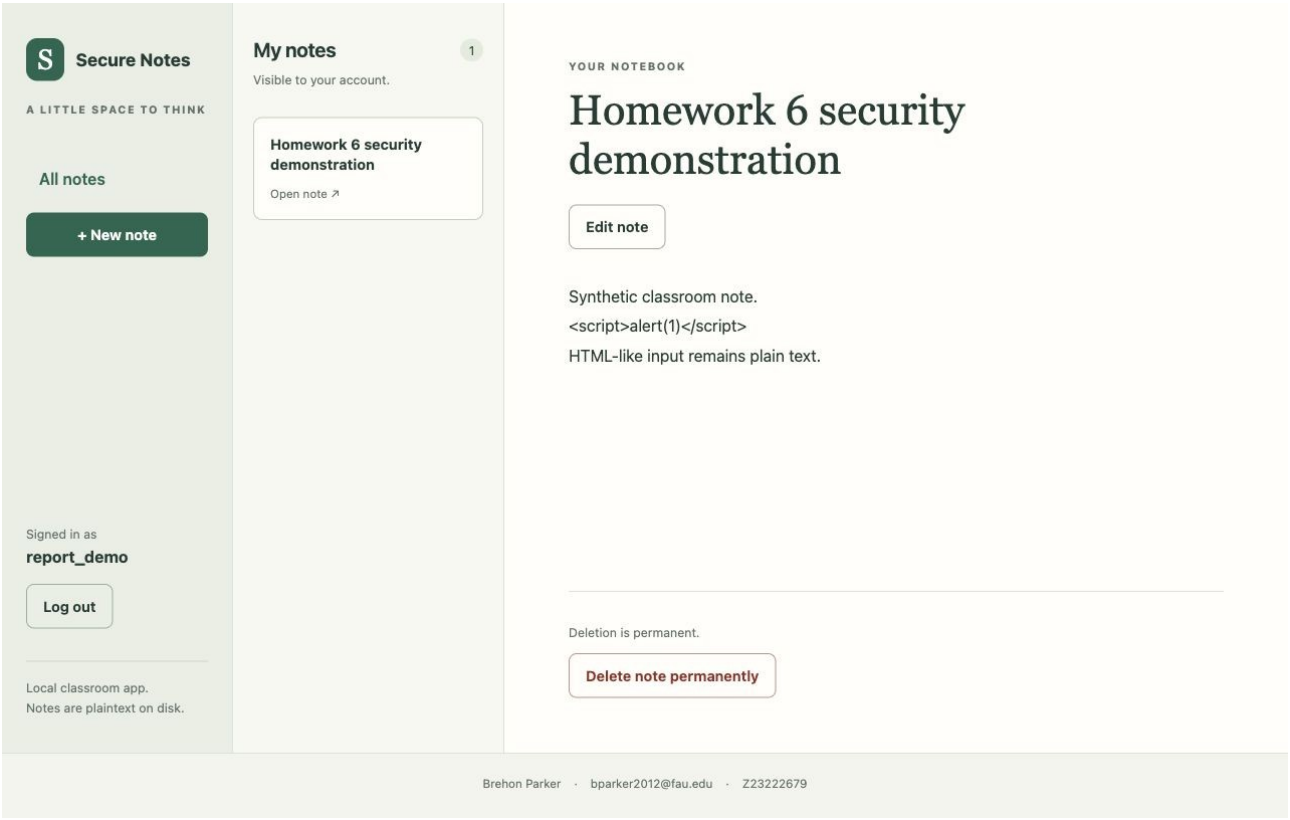


Figure 10. Test note with script-like input displayed as text and the FAU email visible in the footer.

Capabilities and limitations

The app supports separate accounts and persistent private notes, with password hashing, parameterized SQL, output escaping, owner checks, CSRF verification, revocable sessions, request limits, and authentication rate limits. Note content is plaintext at rest. Filesystem access exposes it, and deleted data may remain in database pages or backups. The app has no MFA, password recovery, sharing, or attachments. Public deployment requires HTTPS, a production server, operational controls, and further security review.

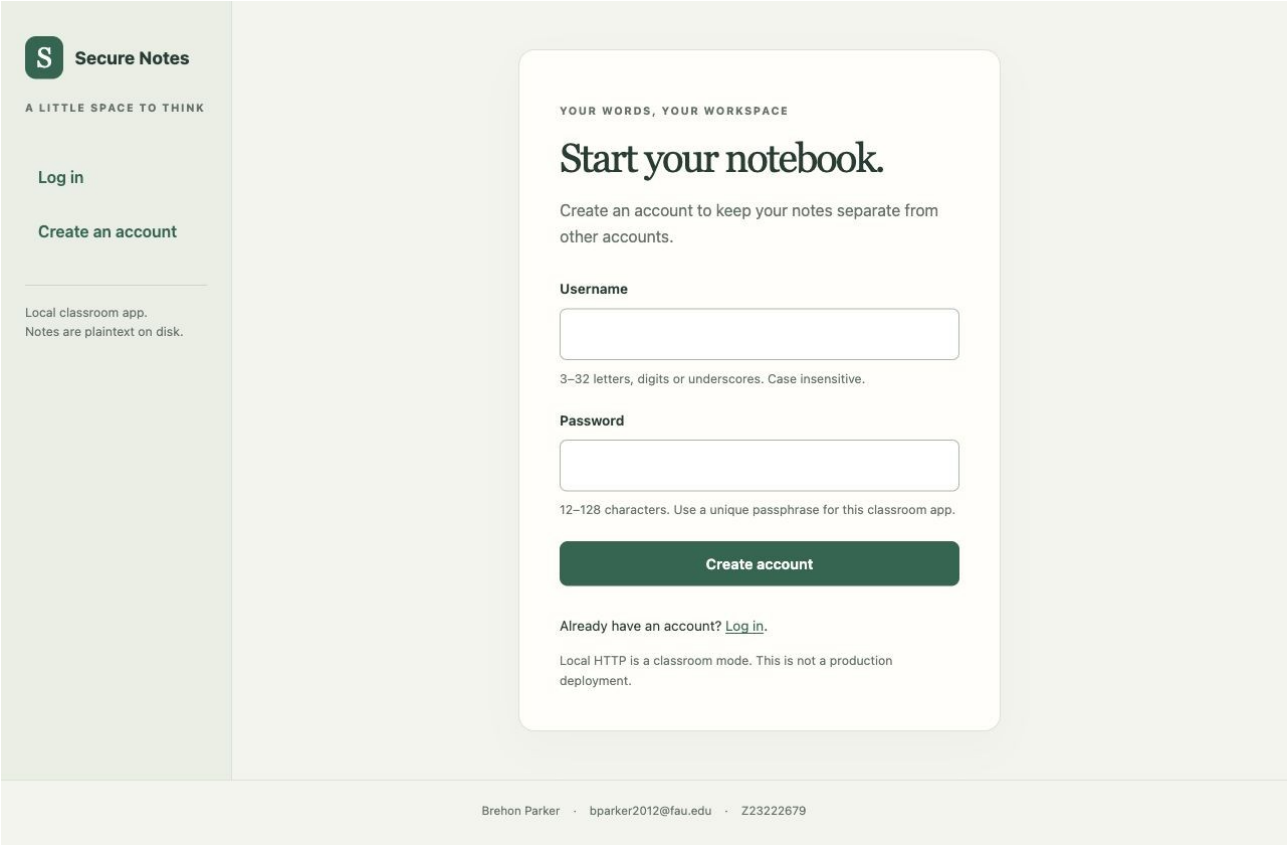


Figure 11. Registration page with the local HTTP and plaintext storage limitations visible.

Current bounds include 120-character titles, 20,000-character note bodies, 96 KiB requests, 1,024 live sessions, and 512 live rate-limit buckets. Each login and registration action allows eight credential submissions per peer IP and normalized account within a five-minute window. There is no total storage quota.